

Day 10 Assignment Report

Name: Asser Mohammed Khairallah

Group Code: GIZ5_SWD5_S2

Assignment: Day 10 - Generics, Delegates, Functional Programming, and Research Topics

Research Topic 1: Parallel Programming and Concurrency

1. Sequential vs. Parallel vs. Concurrency:

- Sequential execution runs instructions one after another with no overlap. Concurrency means making progress on multiple tasks within overlapping time, even on a single core. Parallelism means physically executing multiple instructions at the exact same moment, which requires multiple CPU cores.

2. CPU cores, multithreading, and multicore processing:

- Modern CPUs have multiple independent cores. Multithreading splits a program into threads the OS schedules across those cores, achieving true parallelism when threads run on separate cores simultaneously.

3. Shared-memory vs. message-based approaches:

- In shared-memory, threads read/write the same data directly and need synchronization. In message-based approaches, units communicate by sending messages instead of sharing memory directly.

4. Race conditions, critical sections, and thread safety:

- A race condition happens when correctness depends on unpredictable thread timing. A critical section is code that must be executed by only one thread at a time. Locks enforce this, and code is thread-safe when it behaves correctly regardless of thread interleaving.

5. Immutability and shared mutable state:

- Immutable data can be read safely by many threads at once without locking, which is why functional programming favors immutability to simplify concurrent code.

6. Task-based parallelism and `Task.WhenAll` :

- The Task Parallel Library represents work as `Task` objects instead of raw threads. `Task.WhenAll` creates a task that completes once all supplied tasks finish, commonly used to run independent operations concurrently.

```
Task<int> t1 = Task.Run(() => Square(4));
Task<int> t2 = Task.Run(() => Square(7));
int[] results = await Task.WhenAll(t1, t2);
```

7. Benefits, limitations, and common mistakes:

- **Benefits:** better hardware utilization, faster CPU-bound workloads.
- **Limitations:** added complexity, harder debugging, synchronization overhead.
- **Common mistakes:** sharing mutable state without locks, over-parallelizing trivial work.

8. When to use parallelism:

- Use it for CPU-bound work that splits into independent units. Avoid it for small workloads or I/O-bound work, which suits asynchronous programming better.
-

Research Topic 2: Unit Testing and Test-Driven Development (TDD)

1. Purpose of unit testing:

- Verifies that a small, isolated piece of code (a method or class) behaves correctly, independent of external systems, catching regressions early.

2. Isolation, determinism, repeatability, and assertions:

- Tests should isolate the unit under test, produce the same result every run, be repeatable in any order, and assert specific, meaningful outcomes rather than superficial conditions.

3. Red-Green-Refactor cycle:

- Write a failing test (red), write minimum code to pass it (green), then improve the code's structure while keeping tests passing (refactor).

4. Benefits and limitations of TDD:

- **Benefits:** encourages small well-defined units, builds a regression safety net, leads to more testable design.
- **Limitations:** can slow initial development and requires discipline to maintain.

5. Arrange-Act-Assert pattern:

- Arrange sets up the test data and dependencies, Act invokes the behavior, and Assert checks the actual result against the expected one.

```
[TestMethod]
public void ApplyDiscount_TenPercent_ReturnsNinety()
{
    var calculator = new DiscountCalculator();
    decimal result = calculator.ApplyDiscount(100m, 10m);
    Assert.AreEqual(90m, result);
}
```

6. Testing delegates, generics, and collection transformations:

- Test delegate-based code by supplying a controlled lambda and asserting on the outcome. Test generic methods with more than one type argument. Test LINQ-style transformations by asserting on the resulting collection's content and order.

7. Common testing mistakes:

- Testing private implementation details instead of observable behavior, writing weak assertions, and letting tests depend on shared state or execution order.

8. Recommended practices:

- Keep tests small, focused, and clearly named; keep them fast; avoid duplicating production logic inside tests.
-

Research Topic 3: Asynchronous Programming with async

and await

1. Why asynchronous programming is needed for I/O-bound work:

- I/O operations (file access, network calls, database queries) spend most of their time waiting. Async programming lets a thread start the operation and return immediately instead of blocking while idle.

2. Asynchronous vs. parallel programming:

- Parallel programming uses multiple cores to speed up CPU-bound work. Asynchronous programming avoids blocking a thread while waiting for I/O-bound work, without necessarily running anything simultaneously.

3. Purpose of async and await:

- `async` marks a method as containing asynchronous logic and enables `await` inside it. `await` suspends the method until the awaited operation completes, without blocking the calling thread.

4. Task-returning asynchronous method model:

- Asynchronous methods typically return `Task` or `Task<TResult>`, representing work that may not have completed yet, which callers can await or compose with `Task.WhenAll`.

5. Cancellation with `CancellationToken`:

- A `CancellationToken` lets a long-running async operation be cancelled cooperatively, typically by throwing an `OperationCanceledException`.

6. Exception handling and the importance of awaiting:

- Exceptions in an async method are captured on the returned `Task` and only surface when it is awaited. An un-awaited `Task` can hide failures silently.

7. Effect on thread usage and responsiveness:

- Because `await` frees the calling thread instead of blocking it, a small thread pool can service many concurrent I/O-bound operations, keeping UI apps responsive and servers scalable.

```
public static async Task<string> FetchDataAsync(CancellationToken token)
{
    await Task.Delay(1000, token);
    return "Data retrieved successfully.";
}
```

8. Common mistakes:

- Fire-and-forget calls without awaiting, blocking on async code with `.Result` or `.Wait()`, and ignoring `Tasks` so faults go unnoticed.

9. Choosing async over parallel programming:

- Use asynchronous programming for I/O-bound work (network, file, database). Use parallel programming for CPU-bound work that can be split across cores. Many applications combine both.

Comparative Summary

- **Parallel Programming:** runs independent computations at the same time across CPU cores;

best for CPU-bound work.

- **Concurrency (general):** manages multiple tasks making progress over overlapping time, even on a single core.
- **Unit Testing / TDD:** a development practice for verifying correctness and driving design, not a runtime execution model.
- **Asynchronous Programming:** avoids blocking threads while waiting on I/O-bound operations.